



Crack the Hash

Offensive Security Write-up Autor: Luisa Luna **Data:** 20 de Abril de 2026

Sinopse

O **Crack the Hash** é um desafio de CTF que explora a quebra de hashes. Possui os níveis 1 e 2, e consiste em pegar os códigos hash dados pelo [TryHackMe](#) para encontrar a partir deles as senhas originais, que seriam as flags.

Nível 1

Primeiro Hash

O primeiro passo, que se repetirá para todas as outras hashes, consiste em utilizar a ferramenta [hash-identifier](#) para tentar descobrir qual é o tipo daquele hash:

```
HASH: 48bb6e862e54f2a795ffc4e541caed4d
Possible Hashs:
[+] MD5
[+] Domain Cached Credentials - MD4(MD4(($pass)).(strtolower($username)))
```

Como mostra a figura acima, o hash foi identificado como um possível **MD5** ou **MD4**. Após isso, coloca-se o código dentro de um [arquivo.txt](#):

```
(kali@kali)-[~]
$ echo "48bb6e862e54f2a795ffc4e541caed4d"> hash1.txt
```

Após isso, procura-se o ID dos possíveis tipos de hash no site [hashcat.net](#), e os testa no comando [hashcat](#). Se ele for capaz de quebrar esse hash e o tipo estiver correto, ele entregará a senha original:



```
(kali㉿kali)-[~]  
$ hashcat -a 0 -m 0 hash1.txt /usr/share/wordlists/rockyou.txt.gz  
hashcat (v7.1.2) starting
```

Nesse caso, o tipo de hash foi confirmado como sendo um MD5, de ID: 0. Assim, foi possível conseguir a primeira senha:

```
48bb6e862e54f2a795ffc4e541caed4d:easy
```

Segundo Hash

Novamente, utilizando o [hash-identifier](#):

```
HASH: CBFDAC6008F9CAB4083784CBD1874F76618D2A97  
Possible Hashs:  
[+] SHA-1  
[+] MySQL5 - SHA-1(SHA-1($pass))
```

Agora seguindo para colocá-lo no arquivo.txt:

```
(kali㉿kali)-[~]  
$ echo "1DFECA0C002AE40B8619ECF94819CC1B" > hash2.txt
```

Depois utilizando o [hashcat](#) e testando os possíveis tipos de hash:

```
(kali㉿kali)-[~]  
$ hashcat -a 0 -m 100 hash2.txt /usr/share/wordlists/rockyou.txt.gz  
hashcat (v7.1.2) starting
```

Dessa vez, ele foi identificado como sendo um do tipo SHA-1, de ID: 100, e novamente a senha original foi descoberta:

```
cbfdac6008f9cab4083784cbd1874f76618d2a97:password123
```

Terceiro Hash



Repetindo o uso do [hash-identifier](#):

```
HASH: 1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032
Possible Hashs:
[+] SHA-256
[+] Haval-256 word length supported by kernel: 0
```

Colocando o código dentro de um arquivo:

```
(kali㉿kali)-[~]
$ echo "1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032"> Hash3.txt
```

Testando o [hashcat](#):

```
(kali㉿kali)-[~]
$ hashcat -a 0 -m 1400 Hash3.txt /usr/share/wordlists/rockyou.txt.gz
hashcat (v7.1.2) starting
```

Foi identificado como um [SHA-256](#) de ID: 1400, e a senha original era:

```
1c8bfe8f801d79745c4631d09fff36c82aa37fc4cce4fc946683d7b336b63032:letmein
```

Quarto Hash

Dessa vez, o [hash-identifier](#) não foi capaz de identificar os possíveis tipos do quarto hash:

```
HASH: $2y$12$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom
Not Found.
```

A saída foi visitar a lista de hashes do [hashcat.net](#), e procurar um que tivesse uma estrutura similar à que estávamos tentando decifrar. Fazendo isso, foi possível identificá-lo como um [bcrypt](#) de ID: 3200.

3200	bcrypt \$2*\$, Blowfish (Unix)
------	--------------------------------



Ao tentar rodar o [hashcat](#), o processo para decodificar o hash estava extremamente lento, então, analisando a quantidade de caracteres indicada na resposta do [TryHackMe](#), utilizamos o comando [grep](#) para criar uma nova lista de palavras menor do que a [RockYou](#), com o objetivo de diminuir o tempo de espera:

```
(kali@kali)-[~]  
$ grep -E '^[a-z]{4}$' /usr/share/wordlists/rockyou.txt > lists.txt
```

No final a senha original encontrada foi:

bleh

Quinto Hash

Nessa parte, o hash-identifier não conseguiu identificar o tipo de hash correto. A solução dessa vez foi visitar o site [dcode.fr](#) para descobrir o seu real tipo (identificado por ele como um MD4) e decodificar o hash diretamente por ele:



A última senha do primeiro nível era:

Eternity22

Nível 2

Primeiro Hash

O processo para conseguir a primeira senha do segundo nível também foi iniciado com o uso do [hash-identifier](#):



```
HASH: F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85
Possible Hashs:
[+] SHA-256
[+] Haval-256
```

Depois, seguiu colocando o código em um [arquivo.txt](#):

```
(kali㉿kali)-[~]
$ echo "F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85" > hash1.txt
```

E novamente, usando o [hashcat](#):

```
(kali㉿kali)-[~]
$ hashcat -a 0 -m 1400 hash1.txt /usr/share/wordlists/rockyou.txt.gz
hashcat (v7.1.2) starting
```

Esse hash também foi identificado como sendo do tipo [SHA-256](#) de ID: 1400, e a senha original encontrada foi:

```
f09edcb1fcefc6dfb23dc3505a882655ff77375ed8aa2d1c13f640fcc2d0c85:paule
```

Segundo Hash

Para o segundo, os tipos encontrados pelo [hash-identifier](#) não funcionaram na hora de executar o [hashcat](#). Então, consultando novamente o [hashcat.net](#) e testando tipos próximos, foi possível descobrir que o hash era um [NTLM](#), de ID: 1000:

Opções encontradas pelo [hash-identifier](#):

```
HASH: 1DFECA0C002AE40B8619ECF94819CC1B
Possible Hashs:
[+] MD5
[+] Domain Cached Credentials - MD4(MD4(($pass)).(strtolower($username)))
```

Parte da lista dos possíveis tipos no [hashcat.net](#):

900	MD4
1000	NTLM
1100	Domain Cached Credentials (DCC), MS Cache



Executando o [hashcat](#) com o ID correto:

```
(kali@kali)-[~]  
$ hashcat -a 0 -m 1000 hash2.txt /usr/share/wordlists/rockyou.txt.gz  
hashcat (v7.1.2) starting
```

E por fim, a senha original descoberta:

```
1dfeca0c002ae40b8619ecf94819cc1b:n63umy8lkf4i
```

Terceiro Hash

O tipo do terceiro também não pode ser identificado com o [hash-identifier](#). Porém, consultando dessa vez ambos os sites [hashcat.net](#) e [dcode.fr](#), foi possível descobrir o terceiro hash como um [sha512 crypt](#), de ID: 1800:

Consulta no [dcode.fr](#):

↑↓	↑↓
algo	SHA-512

Consulta no [hashcat.net](#):

1800	sha512crypt \$6\$, SHA512 (Unix) ²	\$6\$qdMgClgO2dQWB37F\$jhexCX1SdsCAI00
------	---	--

E novamente para acelerar o processo de quebra da hash, foi utilizado o [grep](#) para filtrar a lista de palavras:

[grep](#):

```
(kali@kali)-[~]  
$ grep -E '^.{6}$' /usr/share/wordlists/rockyou.txt.gz > list.txt
```

[hashcat](#):

```
(kali@kali)-[~]  
$ hashcat -a 0 -m 1800 hash3.txt list.txt  
hashcat (v7.1.2) starting
```

E a terceira senha:



```
$6$aReallyHardSalt$6WKUTqzq.UQQmrm0p/T7MPpMbGNnzXPMAXi4bJML9be.cfi3/qxIf.hsGpS41BqMhSrHVXgMpdjS6xeKZAs02.:waka99
```

Quarto Hash

Para conseguir a quebra do último hash, novamente o [hash-identifier](#) não encontrou o tipo correto, mas consultando o [hashcat.net](#) e testando tipos próximos, foi possível descobrir que este era um **HMAC-SHA1**, de ID: 160, que precisava de uma chave **salt** ([tryhackme](#)) inserida no final do código. Esta foi dada pelo próprio TryHackMe:

[hash-identifier](#):

```
HASH: e5d8870e5bdd26602cab8dbe07a942c8669e56d6
Possible Hashs:
[+] SHA-1
[+] MySQL5 - SHA-1(SHA-1($pass))
```

Tipos testados do [hashcat.net](#):

110	sha1(\$pass.\$salt)
120	sha1(\$salt.\$pass)
130	sha1(utf16le(\$pass).\$salt)
140	sha1(\$salt.utf16le(\$pass))
150	HMAC-SHA1 (key = \$pass)
160	HMAC-SHA1 (key = \$salt)

[hashcat](#):

```
(kali@kali)-[~]
$ hashcat -a 0 -m 160 Hash4.txt /usr/share/wordlists/rockyou.txt.gz
hashcat (v7.1.2) starting
```

E para finalizar o CTF, o último hash quebrado:

```
e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme:481616481616
```